

北京理工大学

本科生毕业设计(论文)

Starry-OS 共享调度器设计与实现

Design and Implementation of Unified Scheduling for Starry-
OS

学 院:	计算机学院
专 业:	计算机科学与技术
班 级:	07112204
学生姓名:	肖嘉骏
学 号:	1120223150
指导教师:	陆慧梅

2026 年 5 月 26 日

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：肖嘉骏

日期：2026 年 5 月 26 日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：①学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；③学校可允许本毕业设计（论文）被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；⑤学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：肖嘉骏

日期：2026 年 5 月 26 日

指导老师签名：陆慧彬

日期：2026 年 5 月 26 日

Starry-OS 共享调度器设计与实现

摘 要

随着高并发异步应用场景的不断增多，传统操作系统中用户态调度与内核态调度相互分离的问题逐渐突出。现有调度机制通常以线程作为内核调度单位，在用户态构建了一套独立于内核的调度体系，产生了调度对象不统一、调度层次分离、任务切换路径割裂三个问题。针对上述问题，本文设计并实现了面向宏内核的共享调度器 vsched2，并对其进行了系统适配与验证。

本文扩展 AsyncOS 的统一任务思想，针对调度对象不统一问题，通过栈池复用栈，将线程与无栈协程统一抽象为任务，并以任务作为统一调度单位；针对调度层次分离问题，采用 vDSO 机制实现跨特权级与跨地址空间的统一调度；针对任务切换路径割裂问题，设计共享的任务切换路径，统一中断上下文和任务上下文的切换过程。此外，本文提出了基于事件源的调度模型，将系统调用、IPC 与中断等统一抽象为事件源，并结合优先级机制实现事件驱动调度。

为了支持共享调度器在用户态与内核态之间共享代码与数据，本文进一步在 Starry-OS 上移植了基于 vDSO 的共享运行机制，并完成了 vdso_crate_template 框架适配。随后，通过 vipc 库对 vDSO 机制进行了验证。最后，本文实现了 vsched2 与 Starry-OS 的接口适配，完成了部分测试工作，后续需进一步完善调试工作。

实验结果表明，本文实现的 vDSO 机制能够在 Starry-OS 上正常运行，共享调度器 vsched2 在 Starry-OS 中能够正常编译，有待进一步测试。相关工作作为后续研究统一调度、用户态运行时以及跨特权级协同调度机制提供了基础。

关键词：统一调度；协程；任务；vDSO；操作系统；Starry-OS

Design and Implementation of Unified Scheduling for Starry-OS

Abstract

With the increasing number of highly concurrent asynchronous application scenarios, the separation between user-space scheduling and kernel-space scheduling in traditional operating systems has gradually become more prominent. Existing scheduling mechanisms usually take threads as the kernel scheduling unit and construct an independent scheduling framework in user space, resulting in three problems: inconsistent scheduling objects, separated scheduling layers, and fragmented task switching paths. To address the above problems, this thesis designs and implements a shared scheduler, vsched2, for monolithic kernels, and performs system adaptation and verification for it.

This thesis extends the unified task concept of AsyncOS. To address the problem of inconsistent scheduling objects, thread stacks are reused through a stack pool, and both threads and stackless coroutines are uniformly abstracted as tasks, which serve as the unified scheduling unit. To address the problem of separated scheduling layers, the vDSO mechanism is adopted to implement unified scheduling across privilege levels and address spaces. To address the problem of fragmented task switching paths, a shared task switching path is designed to unify the switching process between interrupt contexts and task contexts. In addition, this thesis proposes an event-source-based scheduling model, which uniformly abstracts system calls, IPC, and interrupts as event sources, and combines them with a priority mechanism to implement event-driven scheduling.

To support code and data sharing of the shared scheduler between user space and kernel space, this thesis further ports a vDSO-based shared runtime mechanism to Starry-OS and completes the adaptation of the vdso_crate_template framework. Subsequently, the vDSO mechanism is verified through the vipc library. Finally, this thesis implements the interface adaptation between vsched2 and Starry-OS and completes part of the testing work, while further debugging and improvement are still required.

Experimental results show that the vDSO mechanism implemented in this thesis can run correctly on Starry-OS, and the shared scheduler vsched2 can be successfully compiled in Starry-OS, although further testing is still needed. The related work provides a foundation for future research on unified scheduling, user-space runtimes, and cross-privilege-level cooperative scheduling mechanisms.

Key Words: Unified Scheduling; Coroutine; Task; vDSO; Operating System; Starry-OS

目 录

摘 要	I
Abstract	II
第 1 章 绪论	1
1.1 研究背景	1
1.2 研究内容	2
1.3 研究目标	3
第 2 章 相关技术与背景	5
2.1 协程模型	5
2.1.1 协程的基本概念	5
2.1.2 Rust 异步模型	5
2.2 用户态与内核态调度	6
2.2.1 内核态调度	6
2.2.2 用户态调度	6
2.2.3 统一调度	8
2.3 vDSO 技术	8
2.3.1 vDSO 的基本原理	8
2.3.2 vDSO 在统一调度中的作用	9
第 3 章 共享调度器设计	10
3.1 总体架构	10
3.1.1 任务保存过程	11
3.1.2 调度过程	12
3.1.3 任务恢复过程	13
3.2 核心抽象设计	14
3.2.1 任务	14
3.2.2 栈与栈池	15
3.2.3 事件源	17
3.3 调度流程	19
3.4 优先级机制	22

3.5 vDSO 机制.....	23
第4章 共享调度器实现	26
4.1 模块划分.....	26
4.2 调度循环实现.....	27
4.2.1 循环内函数跳转链	27
4.2.2 对外界操作系统约定	28
4.2.3 核心算法实现	29
4.3 栈管理.....	30
4.3.1 空栈与非空栈	30
4.3.2 栈池管理	31
4.3.3 跳板函数	31
4.4 Starry-OS 适配实现.....	33
4.4.1 适配目标	33
4.4.2 vDSO 移植实现	34
4.4.3 vsched2 接口实现	35
第5章 实验与分析	37
5.1 vDSO 测试.....	37
5.2 vsched2 测试.....	38
结 论	40
参考文献	41
致 谢	43

第1章 绪论

1.1 研究背景

随着高性能网络设备与多核处理器的快速发展，计算机系统面临的高并发、低延迟的场景日益增多。在网络服务器、分布式云计算等领域中，计算机系统需要同时处理大量请求，在几十毫秒的时间尺度上对用户的操作作出响应^[1]。而传统以线程为调度单位的调度模型存在上下文切换开销较大、运行时占用资源多等问题，从而无法满足高并发、低延迟场景下的性能需求。

协程具有运行时占用资源少、切换开销小等特性，在高并发场景下较传统线程切换更具有优势^[2]。在现代编程语言与运行时系统中，协程已经得到广泛应用。例如，Rust 语言通过 `async/await` 语法与 `Future` 模型实现无栈协程机制，能够以较低开销实现大规模异步任务调度；Go 语言通过 `Goroutine` 实现用户态线程模型。

异步运行时是在用户态管理协程生命周期和执行调度的框架。通过将大量用户协程映射到少量内核线程中，形成 `M:N` 关系，从而支持较少开销运行大量协程任务。多种编程语言均开始在运行时层面引入轻量级并发与用户态调度机制。例如 Tokio 异步运行时构建了通过事件驱动的高并发执行框架；Go 语言通过 `goroutine` 与 `G-M-P` 调度模型实现用户态任务调度。此外，Java Loom^[3]、Node.js libuv^[4] 也有协程相关实现方案。

然而，现有协程运行时在用户态构建了一套独立于内核的调度体系，调度机制通常与操作系统内核调度器相分割。用户态线程调度与协程调度因不需要切换地址空间具有调度开销小、速度快的优势，但难以直接感知内核态任务状态；传统内核态调度能够统一管理各种操作系统资源与进行中断处理，但内核级线程切换开销较高。在当前多数操作系统设计中，用户态调度与内核态调度相互分离，虽能在同地址空间的简单切换时降低用户态任务调度的开销，但会带来以下的问题：

- 调度对象不统一：线程与协程缺乏统一抽象，采用独立的调度与上下文管理机制；
- 调度层次分离：内核态和用户态分别维护独立调度体系，缺乏统一的优先级与协同调度机制；

- **任务切换路径割裂：**Trap与任务切换采用独立上下文保存与恢复流程，导致任务切换路径割裂，任务中断与调度相分割。

近年来，一些研究尝试实现统一进程、线程、协程调度。AsyncOS^{[5][7]} 将线程与协程看作为同一级，抽象成“任务”概念而作为调度对象参与调度，并借助定制的硬件 TAIC 实现跨用户态与内核态的统一调度。异步化的高性能微内核架构 ReL4^[6] 在微内核中借助 TAIC 和用户态中断技术实现了异步通知机制。这种使用定制硬件的方案能够在用户态完成部分任务切换，融合了内核级线程与用户级线程的优势，从而减少传统线程切换带来的开销。然而，该方案依赖专用硬件，其可移植性与跨操作系统兼容性较差。

Starry-OS^[8] 是一个基于 Rust 语言实现的模块化开源操作系统，具有较为完善进程与线程管理、系统调用、中断处理的实现。其原有调度机制主要以线程为调度单位的内核态调度，引入协程的支持较少，异步机制较为基础。其代码模块化程度高，代码可维护性强，适合作为异步化研究的基础。

因此，本文主要研究如何以软件形式实现协程、线程、进程统一调度，统一用户态调度和内核态调度，形成一个独立于操作系统的调度框架，并移植到 Starry-OS 中，从而提升 Starry-OS 在高并发场景下的异步性能。

1.2 研究内容

针对现有调度模型中用户态与内核态相分离的问题，本文扩展了 AsyncOS 的设计思想，设计并实现了共享调度器 vsched2，主要研究内容包括以下几个方面。

（1）针对调度对象不统一：利用栈池复用栈，统一线程和协程为任务，作为调度对象

参考 AsyncOS 的设计，将线程与无栈协程视为同一级，抽象为“任务”概念，并以任务作为统一调度单位，实现跨特权级与跨地址空间的统一调度。任务通常以协程状态运行，其运行时的上下文由 Rust 编译器保存在栈上，在让权时从栈上清除，从而实现多个任务复用一個栈；在发生中断或以线程形式被调度时，该任务占用其使用的栈，再分配一个新的栈为其他任务使用。

通过栈池，减少复用栈的开销，并管理栈空间使用。在需要进行栈分配和回收时，从栈池中取出与放回，从而减少栈分配开销。同时通过管理栈池的大小，间接管理栈空间的使用情况。

（2）针对调度层次分离：利用 vDSO 机制在用户态和内核复用相同的调度框架，统一用户态和内核态的调度

借助虚拟动态共享对象（vDSO）机制，实现内核与用户态共享同一份代码和数据，从而保证 vsched2 可以既在内核态运行，又能在用户态运行，且保证两侧的访问的数据同步。共享的内容包括调度器代码、共享数据；不共享的内容是各地址空间私有数据。

在用户态运行时，vsched2 只能进行当前地址空间的调度，而不能对其他地址空间以及内核态的任务直接调度。当调度模块指定的下一个任务不在当前地址空间或不是用户态任务时，vsched2 会进入内核完成地址空间切换与恢复任务上下文。

（3）针对任务切换路径割裂：利用共享的任务切换路径，统一中断上下文和任务上下文的切换过程

任务上下文与中断上下文具有相似性，都以寄存器上下文为主，区别在于寄存器上下文是否完整。中断上下文保存了完整的寄存器内容，可以在中断返回时直接回到原处继续执行；任务上下文只包括被调用者寄存器，需要借助函数调用返回时恢复剩余寄存器。

vsched2 把中断上下文和任务上下文的切换统一为一个上下文切换接口，并通过 vDSO 跨地址空间和特权级进行共享。对于中断上下文，系统在中断入口保存完整寄存器状态，并在中断返回前调用上下文切换接口，在内核态恢复其上下文；对于任务上下文，则仅保存被调用者寄存器，在恢复上下文时通过上下文切换接口恢复执行状态。

（4）移植共享调度器 vsched2 库到 Starry-OS

将共享调度器 vsched2 移植进 Starry-OS 中，验证共享调度器的有效性和对宏内核的支持。

1.3 研究目标

本文旨在扩展 AsyncOS 的设计理念和思路，将共享调度器的设计扩展到宏内核，实现用户态与内核态统一调度，并在 Starry-OS 上验证其可行性。研究目标包括以下几个方面：

实现统一任务抽象：通过任务抽象，将线程、协程统一为同一级调度对象，从而实现统一调度模型，并减少不同执行模型之间的切换开销。

实现用户态与内核态统一调度：借助 vDSO 机制，使统一调度框架能够同时运行于用户态与内核态，并共享同一份调度数据与调度逻辑，从而实现用户态与内核态之间的统一调度。

降低上下文切换与栈管理开销：通过协程、共享栈池与栈复用机制减少传统线程切换与频繁栈分配带来的额外开销，提高任务切换效率，并降低系统在高并发场景下的资源消耗。

实现调度框架与操作系统解耦：模块化接口设计，将统一调度框架与具体操作系统实现进行解耦，使 vsched2 能够作为独立框架运行，并支持不同操作系统进行适配与扩展，从而提高系统的可移植性与可扩展性。

本文最终希望通过在 Starry-OS 中移植 vDSO 并适配 vsched2，验证共享调度器在实际宏内核操作系统中的可行性，并为后续用户态中断、多核调度与异步运行时扩展提供基础。

第 2 章 相关技术与背景

2.1 协程模型

2.1.1 协程的基本概念

由于线程模型共享进程的资源，线程之间需要额外的同步互斥机制来避免数据竞争，并且线程在运行中独占一个栈空间，因此在大规模的并发编程中，多线程模型体现出诸多不足^[11]。

协程（Coroutine）是一种轻量级用户态执行单元，其概念最早由 Melvin Conway 于 1963 年提出^[12]。其核心特点在于协程能够在特定位置主动让权让出 CPU，并在恢复运行时继续从先前暂停的位置执行。与传统线程相比，协程通常不依赖操作系统内核进行调度，而是在当前进程下的用户态进行调度，因此具有上下文切换开销较小、内存资源占用较低等优势。

根据上下文保存方式的不同，协程通常可分为有栈协程与无栈协程两类。

有栈协程为每个协程分配独立栈空间，其运行方式类似于用户级线程，能够在任意函数位置挂起与恢复^[13]。本质上，有栈协程切换时保存和恢复的上下文是其运行时寄存器上下文和栈。但由于进行切换时要对栈进行保存与恢复，并带来页表刷新的开销，其切换开销相对较大。

无栈协程不单独维护完整的函数调用栈，而是通过状态机保存执行状态。其运行时上下文通常由编译器自动生成并保存在协程对象中，从而减少栈空间占用并降低切换开销^[14]。在无栈协程发生切换时，无栈协程在让出点主动让出，进行状态机的状态转换，最后协程调用返回，从而实现不占用栈空间。现代异步运行时大多采用无栈协程模型。

本文中的任务主要以无栈协程形式运行，从而实现栈的复用，降低协程切换时的开销。

2.1.2 Rust 异步模型

Rust 语言提供了基于 `async/await` 的异步编程模型。Rust 异步编程模型基于 `Future Trait` 实现，异步任务只有在执行器调用 `poll` 时才会继续推进^[15]。Rust 编译器会把 `async` 函数编译为具有几个 `enum` 类型的状态的状态机，不同状态对应协程

执行过程中的挂起点，在此处进行状态的切换和函数返回。因此 Rust 的异步协程属于典型的无栈协程实现^[16]。

Future 是 Rust 异步模型中的核心抽象，表示一个还未完成，但能保证未来的某个时间会完成的操作。Future 通过 poll 函数驱动执行，每次调用 poll 时，Future 会尝试继续执行任务。若任务暂时无法继续运行，则返回 Pending 状态并等待后续再次唤醒；若任务完成运行，则返回 Ready 状态并携带返回值返回。

通常，Rust 异步运行时通过处在用户态的 Executor 在不进入内核的情况下调度 Future，同时使用 Waker 机制实现事件驱动唤醒。当期望的事件发生时，运行时会调用 Waker 将 Future 重新加入调度队列，从而继续执行。

相比传统线程模型，Rust 异步模型具有以下优势：

- 调度无栈协程开销较小，能够以较少线程支持大量并发任务；
- 编译器自动生成状态机，减少手动管理上下文的复杂度；
- 资源利用率高，更适合高并发异步场景。

然而，Rust 异步运行时通常运行于用户态，与操作系统内核调度器相互独立，难以实现用户态与内核态之间的统一调度。

2.2 用户态与内核态调度

2.2.1 内核态调度

传统操作系统主要采用内核态进行线程调度的调度模型。内核调度器负责线程创建、线程切换以及处理器资源分配等工作，并通过时钟中断实现抢占式调度。

内核态调度能够统一管理系统资源，并能够直接处理系统调用、中断与异常等事件，因此对操作系统的资源有较强的控制。同时，内核态调度能够保证不同进程之间的隔离性与公平性。

然而，内核态线程切换通常需要保存与恢复完整寄存器上下文，并涉及特权级切换与地址空间切换，因此其开销包括了寄存器维护、页表刷新、处理器特权级修改，其切换开销相对较高。在高并发场景下，大量线程切换可能带来明显性能损耗。

2.2.2 用户态调度

用户态调度通常由用户态运行时负责，其调度对象一般为协程或用户级线程。由于调度过程无需进入内核，且无地址空间的改变，因此其上下文切换开销较小^[17]。常见的异步运行时列举如表 2-1 所示：

表 2-1 常见异步运行时列举表

运行时	语言	协程类型	调度模型
Tokio	Rust	无栈协程 <code>async/Future</code>	事件驱动 + work-stealing 多线程调度
Go Runtime	Go	有栈协程 <code>Goroutine</code>	GPM 模型： <code>Goroutine</code> 映射到内核线程池
Java Loom	Java	有栈协程 <code>Virtual Thread</code>	JVM 用户态调度，阻塞时自动 unmount 底层 OS 线程
Node.js (libuv)	JS/TS	事件回调与 <code>Promise</code>	单线程事件循环 + 异步 I/O

Go 语言在语言运行时中内置了用户态调度器，通过将用户态任务 `goroutine` 映射到一组内核线程上执行，实现了轻量级并发调度。调度器在用户态完成任务创建、挂起与恢复，从而避免频繁的内核线程切换开销。在高并发场景下，的创建成本远低于线程，使 Go 能够高效支持大量并发任务^[18]。

在 Rust 语言的生态中，Tokio 是最主流的异步运行时，用于构建高性能、可靠的网络和并发应用。以 Tokio 为例，在高并发网络场景下，Tokio 能够支持百万级连接，并维持较低的资源消耗。公开 benchmark 表明，在 10000 并发连接场景下，传统线程模型可能消耗约 20GB 内存，而 Tokio 异步模型仅需约 500MB^[19]。

用户态调度也存在一定局限：

- 无法直接获取内核态任务的状态；
- 无法统一处理中断与异常；
- 系统调用和外部中断仍需要进入内核态处理。

此外，由于用户态调度与内核态调度相分离，虽能在同地址空间的简单切换时降低用户态任务调度的开销，但会带来以下的问题：

- 调度对象不统一：线程与协程缺乏统一抽象，采用独立的调度与上下文管理机制；

- 调度层次分离：内核态和用户态分别维护独立调度体系，缺乏统一的优先级与协同调度机制；
- 任务切换路径割裂：Trap与任务切换采用独立上下文保存与恢复流程，导致任务切换路径割裂，任务中断与调度相分割。

2.2.3 统一调度

为了同时利用用户态调度与内核态调度的优势，一些研究开始探索统一调度机制。统一调度的核心思想是：在保留用户态轻量级调度能力的同时，使用户态任务能够与内核态调度器共享调度信息，从而减少传统调度模型中重复切换地址空间与特权级带来的开销。

目前，Tokio、Go Runtime 等运行时系统虽然实现了高效用户态调度，但其调度器仍然独立于操作系统内核。开源异步单内核操作系统 AsyncOS^[5] 对于用户态与内核态调度分离产生的三个问题，给出了如下解决方案：

1. 针对调度对象不统一：将线程和协程视为同一级，统一为任务，做为调度对象参与调度；
2. 针对调度层次分离：借助定制硬件TAIC，在内核态和用户态共享相同的调度机制；
3. 针对任务切换路径割裂：在TAIC中接入中断信号，并完成中断保存和恢复，从而统一中断和任务切换。

但 AsyncOS 存在一些局限。由于 AsyncOS 是单内核，其地址空间切换和特权级切换机制比较基础，缺少进程切换与特权级切换的实现。其共享调度器无法兼容宏内核。

2.3 vDSO 技术

2.3.1 vDSO 的基本原理

Linux 为减少频繁系统调用带来的用户态与内核态切换开销，引入了虚拟动态共享对象（vDSO）机制^[20]。vDSO 是一种由操作系统内核映射到用户地址空间中的 ELF 动态共享对象，其本质是一段由内核提供的用户态代码与数据页面，用于在用户态直接执行部分系统调用，从而减少频繁陷入内核带来的额外开销。其主要包括两部分：

- 虚拟动态共享对象（vDSO）：主要存储共享的可执行代码；
- 共享数据页（vVAR）：主要储存 vDSO 访问的数据。

在内核进行初始化时，主要进行共享物理页面的分配，记录代码和数据在内核的地址，以及其虚拟地址映射；在用户态初始化时，首先在内核执行 `execve` 系统调用时，将 vDSO 的页面映射到用户地址空间，并把 vDSO 的地址记录在用户栈的动态链接时辅助信息向量（`auxv`）中，用户程序可通过辅助信息向量中的 `AT_SYSINFO_EHDR` 获取 vDSO 的 ELF 基址，并以动态链接方式调用相关函数 [21]。

Linux 系统中，vDSO 通常用于时间获取等高频系统调用。例如 `gettimeofday` 等函数可以直接通过 vDSO 在用户态读取内核维护的数据，而无需真正进入内核态，从而实现用户态与内核态能够共享部分代码与数据。

2.3.2 vDSO 在统一调度中的作用

本文中的共享调度器 `vsched2` 借助 vDSO 机制实现统一调度框架在用户态与内核态之间的共享。这里对 Linux 的 vDSO 机制进行了扩展，不仅仅支持部分系统调用，而是将自定义的函数与其执行需要的数据也进行共享，从而实现 `vsched2` 的将同一份代码在不同特权级运行的特性。

此外，与 Linux 的 vDSO 不同的是，`vsched2` 使用的 vDSO 中，vDSO 的代码段存储共享的可执行代码，vVAR 中存储共享的数据，而 vDSO 的数据段中存储各地址空间私有的非共享数据。

通过 vDSO 映射，用户态能够直接运行部分调度逻辑，并访问统一调度框架中的共享数据结构。当需要切换地址空间或运行内核态任务时，再进入内核完成对应操作。这种方式能够减少用户态调度过程中的内核态切换次数，从而降低调度开销。

第3章 共享调度器设计

在设计阶段中，本文作者与北京理工大学硕士研究生罗熙共同合作，完成设计工作。本章侧重描述共享调度器的总体设计思路。

3.1 总体架构

本文设计了共享调度器 vsched2 的总体架构。整体运行过程可分为三个部分：任务保存、调度过程以及任务恢复，其总体架构如图 3-1 所示。

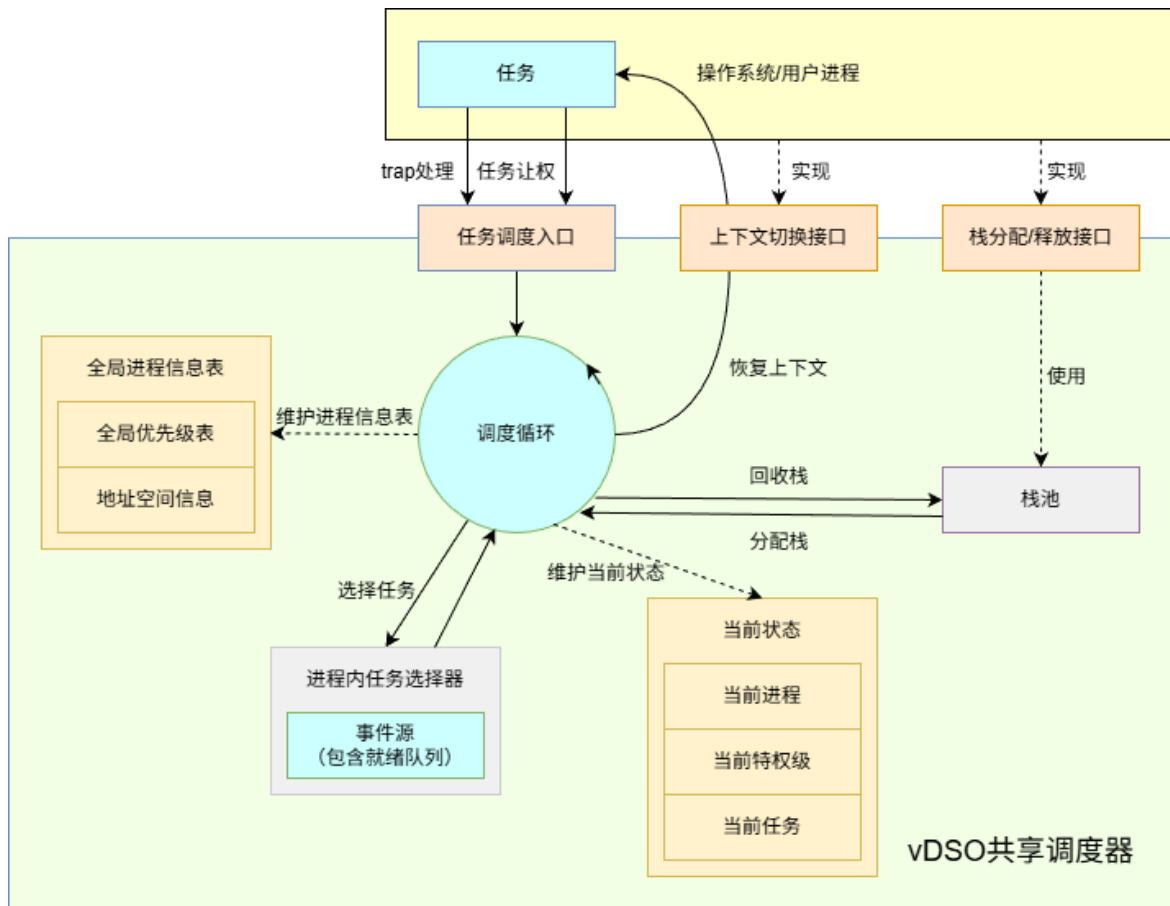


图 3-1 vsched2 总体架构图

在任务运行过程中，当任务主动让权、发生 Trap、以线程形式让权或需要切换地址空间时，系统进入任务保存阶段；随后由调度器，即 vsched2 的调度模块，根据优先级选择下一步运行的任务；最后恢复对应任务的上下文并继续运行。整个过程形成一个循环结构，从而实现统一调度。

3.1.1 任务保存过程

当当前任务发生主动让权、线程形式让权、中断或异常时，`vsched2` 首先需要保存当前执行状态。根据任务当前运行形式的不同，其上下文保存方式也有所区别。

当任务以协程形式运行时，其运行以状态机形式保存在协程对象内部，局部变量与执行状态由 `Future` 对象维护。在任务主动让权后，其调用栈会被清空，并根据返回结果设置任务状态，此后该栈可以被其他任务复用。在发生异步系统调用时，用户态任务也可以被认为是主动让权，然后通过调度器切换到内核态的异步系统调用处理程序，因此异步系统调用时的保存过程与协程让权相同。

当任务以线程形式让权时，需要保留当前栈上的上下文信息。此时，该任务会独占当前使用的栈，并将线程使用的寄存器信息以线程上下文的形式保存在栈中。这里约定操作系统需要完成线程上下文的保存，并跳转到调度框架的指定位置 `thread_entry` 处。同理，线程形式的异步系统调用的保存过程与线程让权相同。此后，调度框架将继续使用这个栈，直到后续运行新任务前才完成新栈的分配。这种延后分配栈的设计的目的是：若下一个任务在内核态运行，延后分配栈可以减少一次栈的保存和恢复过程。

在发生 `Trap` 时，`vsched2` 分三种方式进行处理：

- 外部中断：设置被打断的任务为就绪，然后对外部中断进行处理，最后返回调度模块继续调度；
- 内部中断与异常：包括系统调用。此时则尝试唤醒在初始化时创建的中断处理任务，如果所有任务都被唤醒则新建一个处理任务。然后跳过调度模块之间运行处理程序。与操作系统约定的处理程序流程图如图 3-2 所示；
- 特定的系统调用：指在 `vsched2` 中为了上述切换地址空间或调度内核任务的情况而进入内核，此时根据进入内核的原因进行地址空间切换或选择内核任务运行。

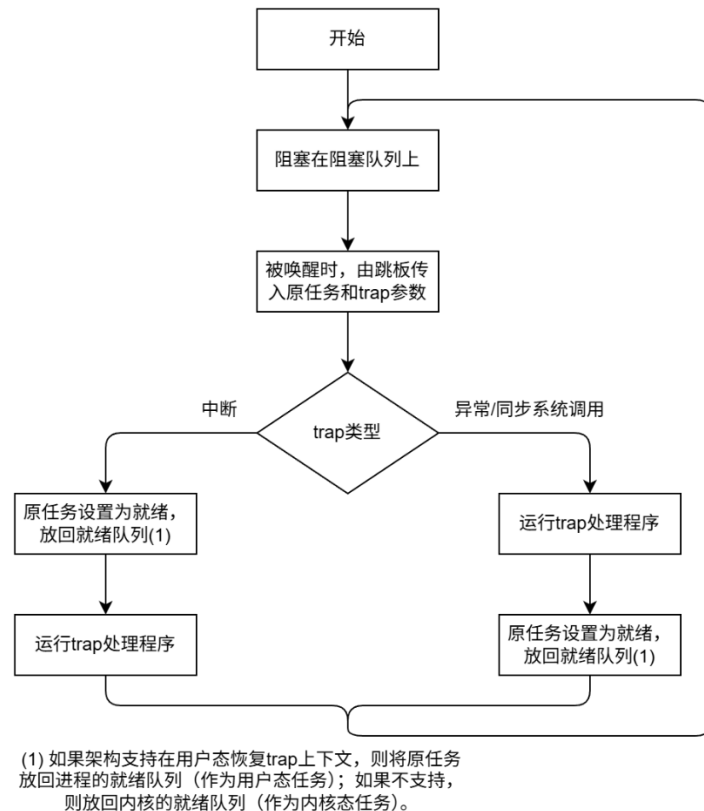


图 3-2 Trap 处理任务流程图

在调度框架进行 Trap 处理前，约定 OS 需要设置中断处理向量中，完成切换预保存栈、保存 Trap 上下文，并设置进入调度框架的 Trap 处理时的参数。

切换预保存栈是指在发生 Trap 时，Trap 处理程序的运行需要占用一个内核栈，例如在 RISC-V 指令集时，这个内核栈通常保存在 sscratch 寄存器中。为了防止 Trap 处理过程中出现新的 Trap，需要为 sscratch 寄存器分配一个新的栈，把这个栈称为预保存栈。此后，Trap 处理程序使用的栈在处理程序返回时回收，新分配的预保存栈则留在了 sscratch 中，供后续的 Trap 处理程序使用。

如果指令平台支持用户态中断，上述过程则可以在用户态完成，与操作系统约定的内容与内核中断相似。

3.1.2 调度过程

在任务调度过程中，vsched2 采用基于优先级的统一调度机制，并以任务作为统一调度单位。

vsched2 中每个地址空间维护自身的最高优先级信息，调度模块通过比较不同地址空间中的最高优先级任务，决定下一步需要运行的任务。优先级相关内容将在后续 3.2 节中详细说明。当调度目标位于当前地址空间时，调度可以直接在当前上下文中完成；若目标任务位于其他地址空间，或目标为内核态任务，则需要进入内核完成地址空间切换与上下文恢复。

由于 vsched2 同时运行于用户态与内核态，因此调度过程可能发生在不同特权级中。当调度框架运行于用户态时，其只能直接调度当前地址空间中的用户态任务；对于跨地址空间或内核态调度，则需要借助 Trap 进入内核态继续完成调度过程，在这个过程中，还会对当前地址空间的任务使用的栈尝试回收，从而维护当前地址空间栈池数量的稳定。

此外，系统调用、中断以及 IPC 等事件会作为事件源参与调度，并能够动态更新对应任务的优先级，从而提高高优先级任务的响应能力。

3.1.3 任务恢复过程

在任务恢复阶段，vsched2 根据当前调度器所处的特权级与目标任务所处的特权级不同，进行不同的恢复操作：

对于当前特权级与目标特权级相同的情况，即前后都属于用户态或内核态，调度框架需区分目标任务是否是协程。如果目标任务是协程，则检查当前调度器使用的栈是否为空栈，空栈不做处理，非空栈则分配一个新的空栈，然后修改目标任务状态为运行，最后调用 Poll 运行协程即可。如果目标任务为线程，则检查当前调度器使用的栈是否为空栈，非空栈不做处理，空栈则回收这个栈，然后修改目标任务状态为运行，最后恢复线程上下文，同时恢复线程栈。

对于当前特权级与目标特权级不同的情况，此指前者属于内核态，后者属于用户态，调度框架也区分目标任务是否是协程。如果目标任务为协程，则为处于用户态的目标任务分配一个用户栈，然后检查当前调度器使用的内核栈是否为空栈，非空栈不做处理，空栈则回收这个栈，最后切换地址空间，在用户态使用刚才分配的栈。如果目标任务为线程，则之间与上述相同尝试回收内核栈，然后修改任务状态为运行，最后切换特权级返回线程内部继续运行。

通过上述任务保存、调度与恢复任务三个阶段，vsched2 构建了一个统一的调度循环，并实现了用户态与内核态、线程与协程之间的协同调度。

3.2 核心抽象设计

`vsched2` 设计了一组独立于具体操作系统与运行时实现的核心抽象，包括任务、栈与栈池以及事件源等概念。通过这些抽象，`vsched2` 能够统一描述不同执行状态下的任务运行过程。

3.2.1 任务

传统操作系统通常以线程作为调度对象，而协程则由用户态运行时独立管理，两者之间缺乏统一的抽象模型。为了实现统一调度，`vsched2` 参考 `AsyncOS` 的设计思想，将线程与协程认为是同一级调度对象，并抽象为“任务”，作为调度框架中的基本调度单位。

任务的本质是在 CPU 中运行的连续的代码序列，并不限定具体运行形式。任务既可以以协程形式运行，也可以以线程形式运行，主要包含以下信息：

1. 任务运行形式：标识运行形式是协程还是线程；
2. 上下文信息：任务运行中的寄存器上下文；
3. 优先级：表示任务的优先程度；
4. 运行状态：包括运行态、阻塞态、就绪态、退出态。

任务运行状态的转换如图 3-3 表示。

任务以协程形式运行时，其任务运行形式会设置为协程，执行状态由 Rust 编译器生成的 `Future` 状态机保存，此时任务能够在主动让权后释放当前栈空间，实现多个任务共享同一个栈。此时其上下文信息的保存与恢复比较简单，除编译器生成的状态机外无需进行其他维护。

当任务以线程形式运行，或在运行过程中发生 `Trap` 时，其任务运行形式会设置成线程，需要保留当前栈中的上下文信息。此时，任务会占用当前使用的栈，并保存寄存器上下文。此时其上下文信息分为了线程上下文与中断上下文，二者的本质区别是线程上下文只维护了被调用者保存寄存器，而中断上下文需要维护所有寄存器。

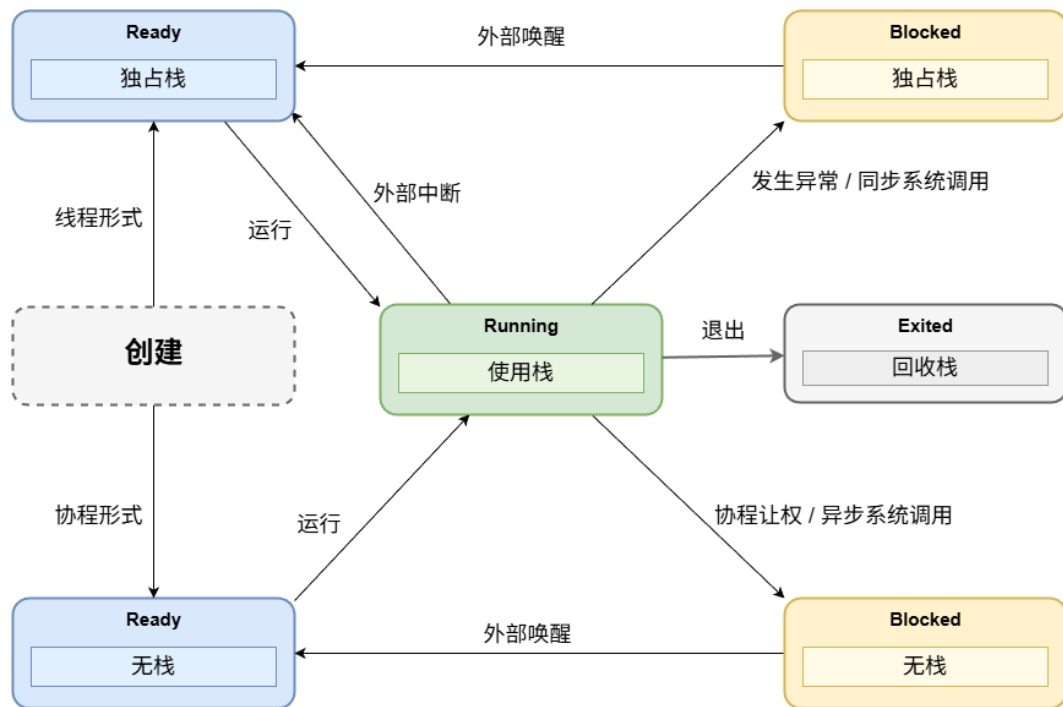


图 3-3 任务状态转换图

此外，每个任务均维护自身优先级与运行状态。其中，运行状态用于描述任务当前是否处于运行、就绪、阻塞或等待事件等状态，优先级描述任务的优先程度。方便调度器的选择任务过程。

在 vsched2 的设计中，地址空间与特权级是独立于任务概念的。vsched2 通过在运行过程中维护地址空间和特权级的状态来判断任务在操作系统中的运行状态，调度框架在进行任务调度的过程中，需要同时进行地址空间和特权级的切换，具体切换流程将在 3.3 节详细说明。

通过任务抽象，vsched2 将传统线程、协程统一纳入同一调度模型，从而实现跨特权级与跨地址空间的统一调度。

3.2.2 栈与栈池

在传统线程模型中，每个线程通常独占一组调用栈，其生命周期与线程绑定。当线程数量较多时，大量栈空间会带来明显内存开销。此外，频繁创建与销毁栈还会带来额外的物理页面分配与虚拟地址映射开销。为了降低栈管理成本，vsched2 采用共享栈机制。在同一地址空间中，多个任务能够共享栈资源。

当任务以协程形式运行并返回调度器时，其使用的栈中没有与任务有关的内容，只可能有一些基本的用户栈信息，可以认为此时的栈是空栈，以供后续调度器和其他

任务复用。如果任务以线程形式返回调度器或被中断，其使用的栈内保留了该任务运行过程中的局部变量、函数调用栈等信息，这时的栈被认为是非空栈，后续任务不能复用，否则会破坏原有任务保存的数据，此时需要在下一个任务运行前，分配一个新的栈，供后续任务使用。

类似的，在任务恢复过程中，也有需要回收栈的情况。例如在线程恢复运行时，如果当前调度器使用的栈为空栈，则需要切换到线程使用的栈，并回收调度器使用的空栈；在内核态调度器选择执行用户态任务时，需要切换到用户态任务使用的栈，并回收调度器使用的内核栈。

在设计中，规定任务在以协程形式进行调度时，均使用空栈；以线程形式进行调度时，均使用非空栈。

当任务发生 Trap 时，Trap 处理程序会占用一个栈运行，在 RISC-V 架构下，这个内核栈存在于 sscratch 寄存器中。此后，如果有新的中断产生，新的中断处理协程也需要占用一个空栈。因此在 Trap 处理之初，需要分配一个新的空栈，供后续中断处理协程使用。

基于上述需求，vsched2 进一步设计了栈池机制，用于统一管理栈资源的分配与回收。栈池的设计与使用如图 3-4 表示。

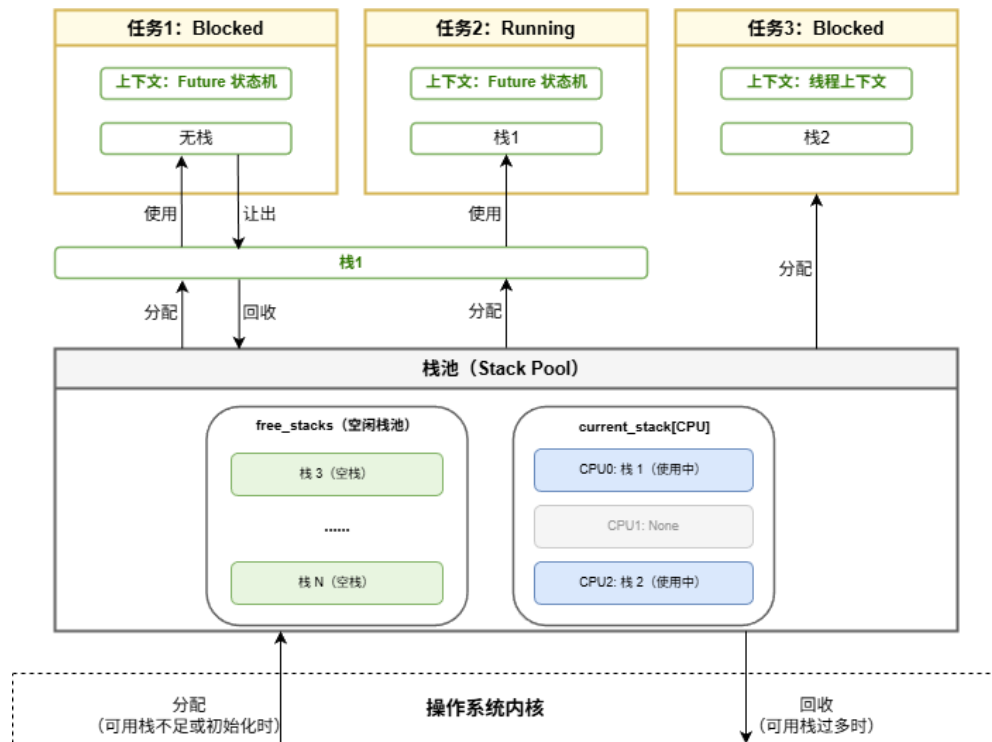


图 3-4 栈池设计图

在用户态中，每个地址空间维护一个独立的栈池；在内核态中，则由所有地址空间共享同一个内核栈池。这样的设计既能够保证不同地址空间之间的隔离性，又能够兼容启用 KPTI 机制^[22]时的操作系统。

栈池中的栈空间通常在初始化阶段统一完成物理页面分配与虚拟地址映射。后续调度过程中只需要从栈池中获取空闲栈即可繁重重新分配和回收页面，从而降低栈分配与回收开销。当栈池中的栈不够时，才进行新栈的页面分配，以满足更多任务的需要；当回收栈时栈池已满，则对栈的页面进行回收，放在栈池大小的不断扩张。在此设计中，最差的情况是所有任务都被中断或以线程方式运行，则每个任务分配一个栈，整体调度机制回退成传统的线程调度。

此外，共享栈机制还能够减少高并发场景下的内存占用。在大量任务以协程形式运行时，系统只需要维护较少数量的实际栈空间即可支持大量任务调度。栈池中的栈在初始化时进行分配。

3.2.3 事件源

在传统操作系统中，中断、系统调用以及 IPC 等事件通常由内核独立处理，而用户态协程运行时则维护自身的事件循环，两者之间缺乏统一抽象。这种设计会导致事件处理与调度逻辑相互分离，从而增加系统切换与协同开销。

vsched2 将事件统一抽象为事件源，并将事件处理过程纳入统一调度框架内部。事件源本质上是将事件转换到到协程模块。当事件发生时，事件源会根据事件类型生成、唤醒或更新对应的处理任务，包括 IPC、系统调用、外部中断等，并使其参与后续调度。

为了统一不同类型事件源的行为，vsched2 为事件源定义了统一接口。事件源需要支持以下两个核心操作：

1. 获取当前可运行协程中的最高优先级；
2. 取出当前最高优先级的处理协程。

其中，获取最高优先级接口用于调度模块快速判断当前事件源中是否存在可运行的任务。若当前不存在可运行协程，则事件源返回一个低于最低调度优先级的特殊优先级，用于表示当前事件源为空。取出协程接口则用于从事件源中获取当前优先级最高的处理协程。该协程在返回后不需要额外初始化即可直接运行。例如，对于需要

向协程传递消息的事件源，其对应消息应在协程返回前已经完成传递。此外，事件源还需要在事件到来时更新其优先级，从而参与当前进程的优先级的更新。

在此基础上，可以将请求-响应队列、协程就绪队列以及中断控制器等模块实现为事件源。

对于带优先级的协程就绪队列，其实现方式最为简单直接。事件源内部维护按优先级排序的就绪协程队列，每次返回当前优先级最高的协程即可。因此，协程就绪队列本身即可视为一种特殊事件源。

对于请求队列，实现为一个协程处理多个请求。例如系统调用处理协程或 IPC 服务协程。请求到达后，请求队列只需要唤醒对应处理协程即可，因此其最高优先级通常由处理协程本身决定。对于响应队列，不同响应通常对应不同阻塞协程，当响应返回时，需要重新唤醒对应阻塞协程。因此，响应队列需要维护响应与协程的对应关系，并按照协程优先级组织响应队列。在唤醒协程前，还需要将对应响应内容传递至协程内部。

对于外部中断控制器，以 RISC-V 架构中的 PLIC 为例，其本身具备维护挂起中断并返回最高优先级中断的能力。因此，PLIC 可以直接映射为事件源：当前挂起中断的最高优先级作为事件源的最高优先级，而对应中断处理协程则作为返回的处理协程。

为了减少频繁遍历事件源带来的额外开销，`vsched2` 为每个事件源维护活跃状态。调度器在获取最高优先级与取出任务时，会自动跳过不活跃事件源，从而降低调度过程中的查询开销。事件源活跃机制主要应用于以下两类场景：

具有额外通知机制的事件源：例如硬件队列或中断控制器。当事件源内部不存在可运行任务时，其可以被标记为不活跃；当新的中断或事件到来时，再重新激活对应事件源，从而避免调度器持续扫描空事件源。

请求队列型事件源：在这种模型中，一个协程通常负责处理整个请求队列中的所有请求。在该协程运行期间，对应事件源可以暂时设置为不活跃；当处理协程重新阻塞并等待新的请求时，再重新激活事件源。

在当前设计中，`vsched2` 内部默认仅将“协程就绪队列”与“中断控制器”实现为核心的事件源，同时提供统一接口允许不同地址空间注册新的事件源，从而提高框架的扩展能力。

通过事件源抽象，`vsched2` 将传统独立的协程调度、IPC、系统调用以及中断处理统一纳入同一调度模型，使系统中的各种事件能够以相似的方式参与优先级调度过程，并形成基于事件驱动的统一调度机制。

3.3 调度流程

`vsched2` 框架总体的运行逻辑是：保存当前任务状态—选择下一任务—恢复目标任务状态，任务在运行中的过程不属于 `vsched2` 的调度流程。由于任务可能运行于不同特权级、不同地址空间，并可能以协程或线程等不同形式运行，因此调度过程中会出现多种不同切换路径。不同路径之间的主要区别在于：

- 当前任务是否以协程形式运行：本质上是当前任务是否持有栈；
- 下一任务是否与当前地址空间相同；
- 下一任务处于用户态还是内核态；
- 是否发生 `Trap`：`Trap` 处理需要涉及跨特权级和跨地址空间切换。

根据切换前后任务所在特权级与任务运行形式的不同，将同地址空间切换时 `vsched2` 的任务切换枚举，如表 3-1 所示：

表 3-1 同地址任务切换枚举

特权级	类型	栈操作	场景
用户态→ 用户态	协程→协程	无（复用栈）	正常协程切换
	线程→协程	保留前任务非空栈， 从栈池分配空栈给后任务	用户代码主动让权、 用户态中断
	协程→线程	回收前任务空栈， 切换至后任务保留的非空 栈	用户态中断返回、 从协程到主动让权的线程
	线程→线程	保留前任务非空栈， 切换至后任务保留的非空 栈	前一个线程主动让权， 后一个线程主动让权结束
用户态→ 内核态	协程→协程	回收用户栈， 分配内核空栈	异步系统调用、 从用户协程到内核协程
	线程→协程	保留前任务栈， 分配内核空栈	同步系统调用、中断、异常、 用户主动让权到内核任务
内核态→ 用户态	协程→协程	回收内核栈， 分配用户空栈	异步系统调用返回、 从内核协程到用户协程
	协程→线程	回收内核空栈， 切换至线程非空栈	同步系统调用、中断、异常返回、 从内核态协程到用户态线程

	协程→协程	无（复用栈）	正常协程切换
内核态→ 内核态	线程→协程	保留前任务非空栈， 分配内核空栈	内核任务被内核中断、 内核任务发生异常
	协程→线程	回收空栈， 切换至线程非空栈	内核任务的中断和异常返回

总体而言，在同地址空间的调度情况中，在不跨越特权级时，只需要考虑任务上下文的保存恢复以及栈的切换，这里的上下文只有协程上下文和线程上下文两种情况，在支持用户态中断的架构下，还存在用户态中断上下文。在跨越特权级时，需要额外进行特权级的切换，以及过程中用户栈与内核栈的分配和回收。特殊的，在发生 Trap 时，由硬件保存中断上下文，进入中断处理程序，被打断的任务上下文恢复与线程上下文恢复认为属于同一个操作，约定由操作系统来实现。

将跨地址空间切换时 vsched2 的任务切换枚举，如表 3-2 所示：

表 3-2 跨地址任务切换枚举

特权级	类型	栈操作	场景
用户态→ 内核态→ 用户态	协程→协程	前任务用户栈回收， 内核栈回收与分配， 后任务用户栈分配	跨地址空间协程切换
用户态→ 内核态 (KPTI)	协程→协程 线程→协程	回收用户栈， 分配内核空栈 保留前任务栈， 分配内核空栈	异步系统调用、 从用户协程到内核协程 同步系统调用、中断、异常、 用户主动让权到内核任务
内核态→ 用户态 (KPTI)	协程→协程 协程→线程	回收内核栈， 分配用户空栈 回收内核空栈， 切换至线程非空栈	异步系统调用返回、 从内核协程到用户协程 同步系统调用、中断、异常返回、 从内核态协程到用户态线程
内核态→ 内核态	协程→协程	无（复用栈）	跨地址空间协程切换

在跨地址空间的调度情况中，除增加了地址空间的切换过程，以及不同地址空间栈的分配和回收，其他过程与同地址空间相似。特殊的，在启用 KPTI 后，进入内核也需要切换地址空间，但栈操作与不启用 KPTI 时相同。

整体来看，vsched2 的各种切换路径虽然切换细节不同，但都可以统一描述为以下过程：

1. 保存当前任务上下文；
2. 更新任务状态；
3. 调用调度模块选择下一任务；
4. 必要时切换地址空间与栈；
5. 恢复目标任务上下文。

下面以两种典型路径为例，对调度过程进行详细说明。

第一种典型路径是同地址空间用户协程到用户协程的切换过程，其对应普通协程让权场景，也是 `vsched2` 中最基础的调度路径。

当当前协程主动让权时，调度器首先保存当前协程状态。由于任务以协程形式运行，其执行状态已经由 `Rust Future` 状态机保存，因此无需额外保存完整调用栈，只需要设置运行状态。然后，调度器将当前任务重新加入就绪队列，并从调度模块中选择下一任务。下一任务同样为当前地址空间中的用户协程，则调度器直接调用其 `Poll` 函数恢复对应协程状态并继续执行。由于整个过程中不涉及 `Trap`、地址空间切换以及线程上下文恢复，因此该路径只需要完成协程状态切换即可。

此路径为调度框架最基础也是最常见的调度路径，其中包括了异步系统调用的情况。过程中不需要进入内核态也不需要切换地址空间，同时不涉及栈的保存与分配，协程上下文的保存和恢复由 `Rust` 编译器完成。因此，该路径能够以较低开销支持大量协程切换，是 `vsched2` 在高并发异步场景中的主要运行方式。

第二种典型路径为“用户任务被中断后切换到内核协程”的过程，其对应系统调用、中断以及异常处理等场景。

当用户任务运行过程中发生中断或异常时，硬件首先自动进入内核态，并保存 `Trap` 上下文。随后，根据 `Trap` 原因设置相应状态参数，并将当前任务设置状态。这部分内容约定由操作系统完成。此后，与传统操作系统直接在当前线程中处理中断不同，`vsched2` 会根据 `Trap` 类型唤醒或创建对应的 `Trap` 处理任务。该 `Trap` 处理任务通常以协程形式运行，可以作为普通任务参与调度。在 `Trap` 原因为异常时或同步系统调用时，可以跳过调度过程，直接运行处理任务。当 `Trap` 处理完成后，调度模块会再次选择下一任务。若需要恢复原线程，则调度器回收当前的内核栈，切换回原线程栈，并恢复此前保存的 `Trap` 上下文，恢复用户态执行。

该路径与传统中断处理流程相比，最大的区别在于：**Trap** 处理被抽象为独立任务。该路径包括了中断、异常、同步系统调用过程。如果架构平台支持用户态中断，则上述路径在用户态中也可以进行，且不需要进行特权级切换。

3.4 优先级机制

为了实现跨特权级与跨地址空间的统一调度，**vsched2** 设计了基于优先级的统一调度机制。该机制不仅需要支持单个地址空间内部的大量协程调度，还需要在多个地址空间以及内核态之间快速决定下一步应该运行的任务。因此，**vsched2** 分事件源优先级、地址空间优先级和全局优先级三个层面组织优先级机制，实现不同粒度下的统一调度，并规定优先级数值越小表示优先程度越高。

事件源优先级是每个事件源维护自身的最高优先级。调度器通过查询所有事件源的最高优先级，从中选择优先级最高的事件源，并从该事件源中取出对应任务运行。若多个事件源具有相同最高优先级，则优先选择切换开销更低的事件源。例如，当前地址空间中的协程就绪队列通常会优先于需要切换地址空间或进入内核态的事件源。这种设计能够在保证优先级正确性的同时，尽量减少上下文切换与地址空间切换开销。

地址空间优先级是在事件源的基础上，每个地址空间以及内核均维护自身的最高优先级，用于表示当前地址空间中所有可运行任务的最高优先级，并通过共享数据结构暴露给其它地址空间与内核态访问。

全局优先级是综合比较各个地址空间优先级后，选择最高地址空间优先级作为全局优先级，同时作为依据选择下一任务。当调度器完成当前地址空间中的优先级计算后，会更新当前地址空间对应的最高优先级。随后，调度器会进一步比较其它地址空间以及内核态的最高优先级。如果内核的优先级最高，则调度内核任务；如果其他地址空间优先级最高，则通过进内核来切换地址空间；否则继续运行本地的任务。

在事件源到来时，对应的事件源优先级会进行更新。在每次调度循环的计算全局优先级时，会根据当前地址空间的事件源优先级与运行任务的优先级更新地址空间优先级，而其他地址空间不会主动更新。其他地址空间的优先级需要在其事件源更新时进行同步更新。

3.5 vDSO 机制

由于 `vsched2` 需要同时运行于用户态与内核态，并在不同特权级之间共享调度逻辑，因此系统需要一种能够在用户态直接执行内核提供代码的机制。为此，`vsched2` 使用虚拟动态共享对象（vDSO）实现内核态与用户态之间的代码与数据共享。

传统 Linux 中的 vDSO 主要用于优化系统调用开销。内核会将部分代码映射到用户地址空间中，使用户程序能够直接调用部分内核提供的功能，而无需频繁陷入内核。然而 Linux 的 vDSO 设计目标是兼容固定系统调用接口，其共享内容通常由内核静态生成，不支持自定义共享代码的逻辑。Starry-OS 原有的 `starry-vdso` 实现主要用于兼容 Linux 接口，仅支持部分系统调用优化，无法将 `vsched2` 通过其进行用户态的映射。

为了实现统一调度框架，本文采用 `vdso_crate_template` 库^[9]提供的 vDSO 框架。与 Linux 的 vDSO 相比，`vdso_crate_template` 库的核心区别在于：允许开发者以 Rust crate 的形式构建完整的共享运行时模块，并同时在用户态与内核态运行。

`vdso_crate_template` 的基本原理与 Linux 的 vDSO 相同，但其将共享的数据分为共享的可变数据和私有的可变数据，前者的修改可以被所有地址空间访问，而后者在所有地址空间中都有一份不同的实现，修改互不影响。`vdso_crate_template` 库使用 vVAR 存储共享的可变数据，而使用 vDSO 中的数据段存储私有的可变数据。

由 `vdso_crate_template` 库构建的 vDSO 总体结构如图 3-5 所示。

在整体结构上，`vdso_crate_template` 将接口划分为“依赖接口”与“提供接口”两类。其中，依赖接口由操作系统实现，用于向 vDSO 提供底层能力。由于 vDSO 本身无法直接完成页表管理、Trap 处理以及地址空间切换等操作，因此这些内容需要由操作系统实现并以依赖接口的形式给 vDSO 使用。此外，`vdso_crate_template` 会把用户编写的希望共享的代码以提供接口的形式向外暴露，操作系统内核或用户库可以直接调用这些接口来运行 vDSO 中的共享代码。

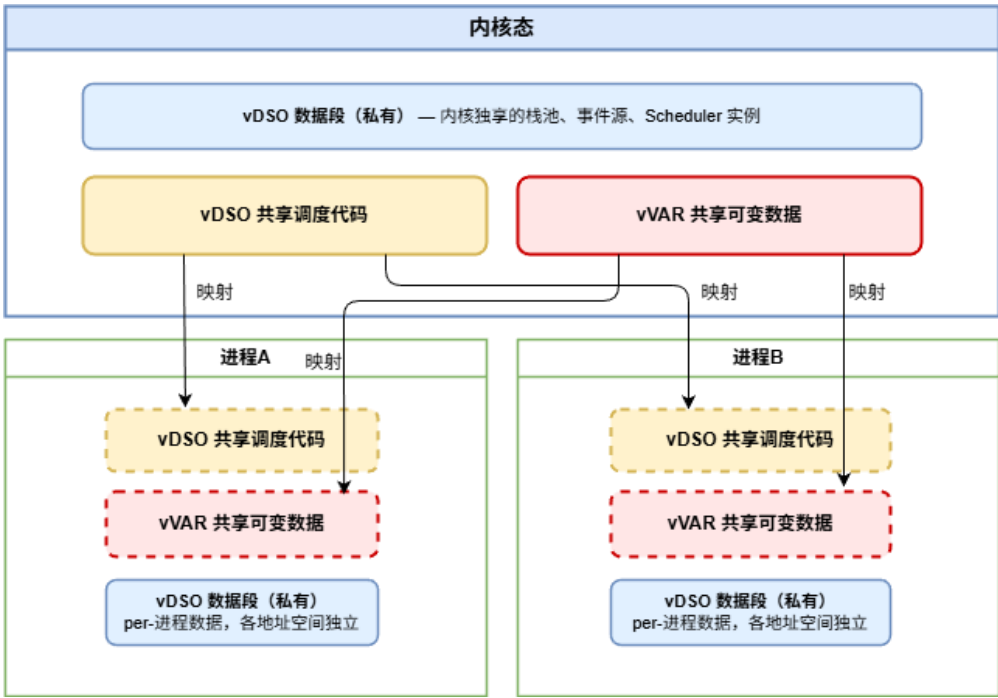


图 3-5 vDSO 结构图

在 vsched2 中，操作系统主要需要提供以下几类依赖接口：

表 3-3 vsched2 的依赖接口

依赖接口	接口详细说明
vDSO 页面映射	由 vds0_crate_template 定义，用于 vDSO 的页面分配页面映射与权限管理
任务定义	为了兼容不同操作系统原本的任务模型，只需要让任务模型实现任务的接口，提供任务的需要的函数
栈管理	分配与回收栈空间
特权级管理	在 vDSO 中无法直接对操作系统的页表寄存器进行修改，因此由操作系统提供特权级切换的实现
地址空间管理	与特权级管理相似，由操作系统提供地址空间切换的实现。
Trap 处理	Trap 发生时硬件直接参与上下文保存，由操作系统提供 Trap 的处理程序，作为任务处理
多处理器管理	在启用多处理器时，操作系统应提供相应的管理函数。

vsched2 主要提供以下接口：

表 3- 4 vsched2 的提供接口

提供接口	接口详细说明
初始化	在操作系统的内核与用户态中调用，初始化 vsched2
提交任务	用于提交任务到就绪队列中
其他	其他操作系统需要的函数与全局变量

通过上述机制，vsched2 能够使用操作系统提供的必要功能，从而实现了跨事件源、跨地址空间以及跨特权级的统一优先级调度，能够在高并发异步场景下动态响应不同类型事件，提高系统整体调度效率与响应能力。

第 4 章 共享调度器实现

在实现阶段中，本文作者与北京理工大学硕士研究生罗熙分工完成开发和测试工作，作者主要负责栈管理、任务保存与恢复、Trap 处理部分。本章侧重描述共享调度器的各模块的结构和关键实现机制。

4.1 模块划分

`vsched2` 作为一个独立于操作系统的统一调度框架，其整体实现采用模块化设计，以降低不同功能之间的耦合，并提高框架的可移植性与可扩展性。根据功能划分，`vsched2` 主要由调度模块、主循环实现模块、全局变量模块、汇编模块以及接口模块五部分组成。

调度模块负责实现调度相关的核心数据结构与算法。该模块主要包括就绪队列、事件源抽象、优先级管理以及调度器实例等内容。调度模块负责从不同事件源中获取当前最高优先级任务，并根据统一优先级机制决定下一步运行对象。此外，该模块还负责维护不同地址空间的最高优先级信息，并实现跨地址空间的优先级比较逻辑。

主循环实现模块用于实现 `vsched2` 的核心调度循环的各个功能。该模块包括 `trap_entry`、`uschedule`、`kschedule`、`run_task` 等关键函数，用于完成 Trap 入口处理、用户态调度、内核态调度以及任务恢复等工作。在组织函数跳转的过程中，`vsched2` 没有使用简单 Rust 函数调用形式，而是通过汇编跳转的跳转指令实现，从而避免编译器因编译优化而带来栈空间异常。在最终实现时，主循环模块以状态机形式组织，各函数对应调度流程中的不同状态，通过跳转形成完整调度循环。

全局变量模块负责维护调度框架运行过程中所需的各种全局状态。由于 `vsched2` 同时运行于用户态与内核态，并在不同地址空间之间共享公共的调度信息，因此该模块需要维护当前任务、当前地址空间、当前栈池以及全局最高优先级等状态变量。这些变量以共享数据的形式存储与 `vDSO` 的 `vVAR` 区域，保证用户态与内核态访问的数据一致性。

汇编模块用于实现不同指令集相关的底层功能，包括调度循环入口、栈切换跳板函数、上下文恢复以及寄存器操作等内容。目前，`vsched2` 已实现 RISC-V 与 x86/x86_64 架构的部分支持。由于不同架构在 Trap 入口、寄存器保存方式以及特权级切换机制上存在差异，因此这些底层操作需要通过架构相关汇编代码实现。

vsched2 通过条件编译对不同架构进行封装,从而保证上层调度逻辑能够以统一方式运行。

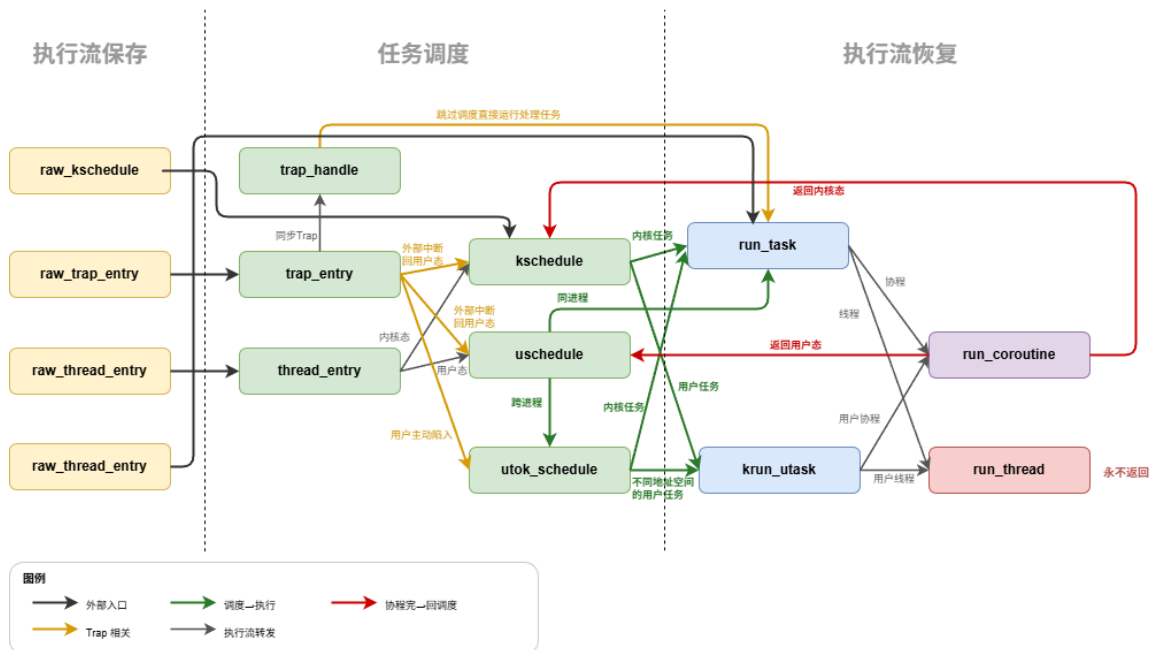
接口模块主要提供了 vsched2 对外暴露依赖接口和提供接口。提供接口包括了内核与用户的初始化与地址空间的生命周期管理接口,用于操作系统在启动阶段完成调度框架初始化,并在进程创建或销毁时完成对应资源管理。依赖接口包括操作系统需要实现的 trait 接口,实现与操作系统解耦,适配不同内核实现。

通过上述模块划分,vsched2 将调度逻辑、架构相关实现以及操作系统相关实现分别进行封装,从而形成一个具有较强可移植性与扩展能力的统一调度框架。

4.2 调度循环实现

4.2.1 循环内函数跳转链

调vsched2 的调度循环由外层汇编大循环实现。在初始化完成后,CPU 会进入调度循环入口,并不断在不同调度状态之间切换。vsched2中的各个函数并不形成普通调用栈,而是由外层汇编代码统一调度任务程。整个调度循环由架构相关汇编代码进入,并在循环中不断调用不同状态对应的函数。每个函数对应调度流程中的一个状态节点,而函数返回值则用于决定下一步跳转目标,因此整体执行过程形成一个如图 4-1 的状态机。



调度循环整体分三个部分：任务保存、任务调度、任务恢复，与图 3-1 整体架构图相对应。在实现中，`trap_entry`、`kschedule`、`uschedule`、`run_task` 等函数与 Trap 入口、内核调度、用户调度、运行任务等状态一一对应。外界操作系统可以调用 `raw_kschedule`、`raw_trap_entry` 等黄色框状态对应的函数进入调度循环，经过 `trap_entry`、`kschedule`、`uschedule` 等绿色框状态对应的函数进行调度，最后由 `run_task`、`krun_utask` 等蓝色框状态对应的函数完成任务执行。例如，当用户态任务发生 Trap 时，硬件触发操作系统的 Trap 处理程序，然后约定操作系统调用 `trap_entry`。该函数完成 Trap 原因分析、上下文保存以及 Trap 处理任务唤醒后，运行处理任务，之后返回 `kschedule` 继续调度。调度器根据优先级选择下一任务，并最终进入 `run_task` 恢复目标任务状态。

4.2.2 对外界操作系统约定

由于 `vsched2` 被设计为独立于操作系统的统一调度框架，因此其本身并不直接依赖具体操作系统实现，而是通过接口与外部操作系统协作完成底层功能。且用户态调度部分需要通过 `vDSO` 与内核共享代码和数据，因此部分底层操作无法完全由 `vsched2` 自身完成，而必须由操作系统提供支持。

在调度流程中，存在一些与操作系统约定，由操作系统实现功能。这些部分在图 3-1 中以虚线框表示。主要分为三部分：

1. Trap 初期处理
2. 外部中断处理
3. 线程上下文保存

在 Trap 初期处理中，由于 Trap 发生时由硬件产生，并同时设置了一些状态寄存器，但 `vsched2` 无法直接读取这些内容。因此约定操作系统应该设置保存 Trap 寄存器、设置预保存栈，并根据状态寄存器的值设置参数，调用 `vsched2` 的 `raw_trap_entry`，进入 `vsched2` 的 Trap 处理函数。

在外部中断处理中，由于外部中断的产生与处理逻辑与操作系统所在的指令平台和外部设备强相关，每个操作系统都有对应的独特的处理逻辑。因此约定外部中断由操作系统处理。在发生外部中断时，应该将被打断的任务设置为就绪，然后进行外部中断处理，最后返回调度器。

在线程上下文保存中，约定当任务以线程形式主动让权或发生线程相关的异步系统调用，应当在用户库中直接实现线程上下文保存，并跳转到 `vsched2` 的 `thread_entry` 函数。这个过程不强制要求进入操作系统内核，但需要在操作系统提供的用户库中进行修改，因此约定操作系统完成。

通过这些接口约定，`vsched2` 能够在不同操作系统之间保持统一调度逻辑，同时避免与具体内核实现强耦合。

4.2.3 核心算法实现

`vsched2` 的调度算法主要包括任务选择、优先级维护和进程-任务两级调度三个部分。

在优先级维护方面，`vsched2` 分事件源、地址空间、全局三层来进行优先级的更新。事件源内部维护自身当前最高优先级，调度器整合当前地址空间中所有事件源的优先级形成当前地址空间优先级，全局进程表 `ProcessInfoTable` 则维护所有地址空间的最高优先级，形成全局优先级。每轮调度循环中，当前调度器都会将自身最高优先级写入全局进程表，其它地址空间则通过读取该表完成跨进程优先级比较。

在具体数据结构上，`vsched2` 的就绪队列 `ReadyQueue` 使用按优先级划分的 FIFO 队列实现。新的任务进入就绪状态时，会被插入对应优先级队列尾部；调度器获取最高优先级时，则从高优先级到低优先级扫描第一个非空队列。由于优先级等级数量固定，因此该扫描过程时间复杂度近似为常数。

在任务选择过程中，每个地址空间对应一个 `Scheduler` 实例。调度器内部维护多个事件源，并通过 `pop_task` 方法完成任务选择。算法首先遍历所有事件源，并获取各事件源当前最高优先级，从中选择优先级最高的事件源。若多个事件源优先级相同，则优先选择索引较小的事件源，以降低跨地址空间切换与跨特权级切换带来的额外开销。然后，调度器调用对应事件源的 `take_task` 接口取出任务，并根据取出任务后的新优先级重新计算当前调度器的最高优先级。

此外，`vsched2` 采用进程-任务两级调度。进程级调度由 `process_schedule` 完成，通过比较所有地址空间中的最高优先级，并决定下一步应切换到的地址空间。任务级调度由 `kschedule` 与 `uschedule` 完成。当目标地址空间为当前地址空间时，调度器能够直接从用户态的本地调度器 `uschedule` 中取出任务运行；而当目标地址空间属于其它进程时，则需要进入内核，通过 `kschedule` 完成地址空间切换，再访问目标进程

调度器。此外，`kschedule` 与 `uschedule` 本身也构成循环结构。若在任务级调度阶段未成功获取任务，例如该任务已被其它 CPU 抢先运行，则调度器会重新返回进程级调度阶段，再次完成全局调度选择。

4.3 栈管理

栈管理是 `vsched2` 实现中的关键机制之一，与任务切换的正确性和性能密切相关。`vsched2` 区分空栈与非空栈两种栈状态，并通过栈池和跳板函数实现栈的分配、回收与切换。

4.3.1 空栈与非空栈

空栈指栈上不包含任何任务相关的有效上下文信息，可以被其他任务和调度循环复用。空栈产生在栈刚分配时和协程返回两种情况。协程在运行期间使用空栈，因为协程的状态由 `Future` 状态机在堆上维护，当协程让出 CPU 并返回时，其运行期间产生的调用栈内容已不再需要。因此协程返回后，该栈恢复为空栈，可以立即被下一个协程复用。

非空栈指栈上保留了任务的有效上下文信息，包括线程的调用栈帧、局部变量和被调用者保存寄存器等。非空栈产生在任务以线程形式让权或被中断时，该任务会独占当前栈，将其变为非空栈。此后调度框架在运行下一个任务之前，需要从栈池中分配一个新的空栈，而保留原有的非空栈在线程上下文中，以在线程恢复运行时重新使用。

总体而言，任务与栈的关系是：栈属于任务的资源，调度器本身只复用前一个任务的栈，栈切换发生在选择的下一任务开始运行前，或切换特权级前后。调度器本身可以复用上一任务的栈，并且只有在下一任务开始运行前，才通过上述 4.2.1 节中的跳板页进行切换，而不是在前一任务返回调度器时就立刻进行切换。这样设计的好处是：当发生用户态任务以协程形式让权，且下一任务处于内核时，如果在上一任务返回调度器时立刻切换栈，会导致栈顶寄存器需要进行两次修改；而放在下一任务运行前切换时则只有一次栈顶寄存器修改。

栈状态的维护通过一个固定的寄存器在整个调度循环中传递。栈状态为 0 表示当前调度器在空栈上运行，不需要在运行下一个任务前回收当前栈；栈状态为 1 表示为非空栈，需要进行回收或保留当前栈。

4.3.2 栈池管理

栈池由 `StackHandler` 结构实现，包含两个数据结构：`free_stacks` 是一个空栈集合，存储已分配但当前不在使用的空栈，即空闲栈池；`current_stack` 是一个 per-CPU 的数组，其本身是 `Option` 类型，记录每个 CPU 当前正在使用的栈，如果为 `None` 说明当前栈池对应的地址空间没有任务正在该 CPU 上运行。

每个地址空间都有一个栈池，通过 `vdso_crate_template` 的全局静态变量实现。内核态共享一个栈池，通过 `vdso_crate_template` 的 `vVAR` 实现。栈池的初始化在进程创建时（`process_init`）和内核初始化时（`kernel_init_main`）进行。初始化时会预分配一定数量的栈，数量为 `STACK_POOL_SIZE - CPU_NUM`，剩下的 `CPU_NUM` 个栈位置留给各 CPU 当前使用的栈。这种预分配能够将物理页面分配和虚拟地址映射的开销集中在初始化阶段完成，后续调度过程中只需从栈池中取出或归还栈即可，避免频繁调用操作系统的页面分配接口。

当需要空栈时，`alloc_stack` 方法优先从 `free_stacks` 池中取出一个栈；如果池已空，则调用依赖接口 `Stack::alloc` 分配一个新栈。当回收栈时，`dealloc_stack` 方法优先将栈放回 `free_stacks` 池中；如果池已满，则调用依赖接口 `Stack::dealloc` 将栈的物理页面和虚拟映射释放。通过这种方式，`vsched2` 在尽可能满足任务对栈的需求的同时，又能维持总体栈数量的稳定。

4.3.3 跳板函数

在整个调度循环中，栈切换是最关键的问题之一，也是调度循环采用状态机形式进行组织的重要原因。

`Rust` 函数调用和返回依赖栈顶寄存器、返回地址寄存器和帧指针寄存器的正确维护，在 `RISC-V` 指令架构中的寄存器分别是 `sp`、`ra` 和 `fp`。`Rust` 的函数调用模型认为在函数运行过程中不会切换栈，且在函数调用链之间栈也不会变化。基于这个函数调用模型，`Rust` 编译器在编译时会存在代码优化、指令重排和寄存器溢出到栈中，并且 `Rust` 内联汇编要求在退出 `asm!` 块时恢复栈指针以及非输出寄存器的原始值，否则将导致未定义行为；但该约束仅在正常退出汇编块时生效，对于不会继续执行或不跳转至其他 `label` 的汇编代码则不适用^[23]。因此在 `Rust` 函数中修改栈是未定义行为，可能会带来未预期的错误，例如编译器生成的帧管理过程中，由于 `sp` 改变导

致在函数序言（`prologue`）中保存的寄存器内容在函数尾声（`epilogue`）中无法访问，从而触发页面异常。

在传统的线程调度模型中，栈切换往往伴随线程切换同时进行，在栈切换的同时将其他被调用者保存寄存器一并切换，这个过程往往在一个裸函数 `switch_to` 中完成。在这种情况下，由于进入 `switch_to` 的函数调用链是固定的，且返回地址寄存器等与函数调用链相关的寄存器也同步修改，因此不会有未定义行为。而 `vsched2` 中，由于任务间栈需要共享，需要能够在运行过程中动态切换栈，容易出现未定义行为，导致编译失败。

为了解决这一问题，`vsched2` 使用跳板函数实现安全的栈切换。跳板函数是一个完全用内联汇编编写的裸函数，没有编译器自动生成的栈帧管理代码。跳板函数的核心操作是：从 `a0` 寄存器读取新的栈指针，将其写入 `sp` 寄存器，然后跳转到目标函数。通过 `switch_sp_trampoline!` 宏生成两种跳板函数：`coroutine_trampoline` 用于切换到空栈后运行协程，`thread_trampoline` 用于切换到线程栈后恢复线程上下文。在 Rust 函数中调用跳板函数时，使用 `jump_to_trampoline!` 宏。该宏首先从帧指针 `fp` 中恢复 `ra` 寄存器的值，然后将新的栈基址传入 `a0` 寄存器，最后由跳板函数跳转到运行任务的运行函数。这里恢复 `ra` 的原因是：在函数调用链中，`ra` 寄存器在函数入口被保存到栈帧中；而跳板函数执行后 `sp` 已改变，不能直接通过 `ret` 返回，因此需要提前从栈帧中取出 `ra` 值，由跳板函数间接返回汇编循环代码。本质上，跳板函数实现了将栈切换操作隔离在一个没有 Rust 栈帧约束的汇编代码块中，使得调用者的栈帧信息和寄存器状态能够安全地切换到新栈上，从而保证了栈切换的正确性。

更进一步，为了让调度框架的代码在运行过程中尽可能避免占用栈空间，需要让每个调度框架的函数都尽可能通过 `return` 的方式退出。因此在最外层使用裸汇编编写了如 `raw_run_task` 的调度循环代码，形成状态机结构的逻辑形式。通过在汇编循环代码中使用 `jmp` 和 `call` 来进行状态切换和具体函数的调用。具体实现逻辑如伪代码 4-1 所示：

表 4-1 栈切换伪代码

1: new_sp	
2: raw_run_task:	汇编裸函数入口
3: call run_task	调用具体函数
4:	

5: run_task:	运行任务的具体函数
6: new_sp ← get_sp()	获取栈顶指针
7: jump_to_trampoline!(coroutine_trampoline, new_sp)	宏跳转到跳板函数
8:	
9: coroutine_trampoline:	跳板函数
10: RA ← FP - 8, FP ← FP - 16	设置 ra 和 fp 寄存器（64 位）
11: SP ← new_sp	汇编设置 sp 寄存器的值
12: j run_coroutine	汇编跳转到运行函数
13:	
14: run_coroutine:	运行协程的函数
15:	具体协程运行的逻辑
16: ret	返回外层汇编循环

这种设计使调度器能够在任意状态安全切换栈，并避免 Rust 编译器生成的隐藏栈状态导致未定义行为。

4.4 Starry-OS 适配实现

4.4.1 适配目标

共享调度器 vsched2 的核心目标之一是实现调度逻辑与操作系统本身的解耦。因此，在完成 vsched2 本身设计与实现后，需要将其适配到 Starry-OS 操作系统中，从而验证其可移植性以及统一调度模型的可行性。

Starry-OS 原有的调度机制主要基于 axtask 模块实现，其调度对象以线程为主，调度逻辑主要运行于内核态。虽然系统在内核中已经具备一定的协程处理逻辑，但用户态任务仍然以线程为调度对象，异步基础较为基础。

与 AsyncOS 相比，在 Starry-OS 中使用共享调度器具有以下难点：

1. 内核架构：Starry-OS 是宏内核操作系统，内核态和用户态的边界明确，有清楚的地址空间区分。需要解决用户态任务访问权限有限、跨地址空间数据不可访问等问题；
2. 任务模型：原有的 axtask 模型比较基础，需要在其上扩展协程支持，同时兼顾线程安全、进程信号等功能正常使用；

3. 硬件依赖: Starry-OS 中没有定制硬件 TAIC, 需要借助 vDSO 机制实现跨地址空间和特权级共享。需要兼顾 Starry-OS 和共享调度器 vsched2 优化 vDSO 机制;

4. 跨指令平台: Starry-OS 支持 x86/64、RISC-V 等多个指令平台, 移植过程需要兼顾各指令平台。

由于 vsched2 依赖 vDSO 机制在用户态与内核态之间共享调度代码与状态, 因此适配工作的第一步是完成 vDSO 的移植, 为后续统一调度框架提供基础运行环境。在此基础上, 还需要为 Starry-OS 原有任务模型实现 vsched2 所需的抽象接口, 包括任务状态获取、上下文切换、栈管理以及地址空间切换等功能, 使 vsched2 能够正确接管原有调度流程。

最终目标是使用 vsched2 替换 Starry-OS 原有 axtask 内部的调度逻辑, 使系统能够以任务为统一调度对象, 并支持协程、线程以及 Trap 处理任务的统一调度。

4.4.2 vDSO 移植实现

为了支持 vsched2 在用户态与内核态共享同一份调度代码, 本文首先在 Starry-OS 中移植了 vdso_crate_template 框架, 从而实现将 vsched2 编译为可映射到用户态的共享代码区域, 并同时在内核态运行对应实现, 为统一调度模型提供基础支持。

在移植过程中, Starry-OS 需要实现 vdso_crate_template 所要求的内存页面加载相关的依赖接口, 包括:

1. `valloc`: 在地址空间分配虚拟内存页面, 且先不分配物理页面;
2. `ppage_alloc`: 分配物理页面;
3. `map`: 映射虚拟内存页面到物理页面;
4. `change_protect`: 调整映射页面的权限;
5. `get_kernel_vaddr`: 获取某地址对应的内核虚拟地址;
6. `ppage_clone`: 复制物理页面对象, 防止 RAII 机制自动回收页面。

在 Starry-OS 中, 对虚拟页面、物理页面的管理分别由 `VirtAddr` 和 `PhysAddr` 这两个结构体定义的, 这两个结构体都有较为完整的实现函数, 并且能与 `u64` 类型之间转换。此外, 这两个结构体没有实现 `drop` 接口, 没有启用 RAII 机制自动回收页面。地址空间的管理由 `AddrSpace` 结构体管理, 具体包括了虚拟地址管理、内存

映射管理和页表管理。该结构体内的函数功能丰富，模块化清晰，能很方便的对内存区域进行操作。

在实现上述六个接口时，使用 AddrSpace 模块的 find_free_area、map_linear、protec 等函数对各地址空间数据结构分配页面并设置权限，使用 global_allocator 模块的 alloc_pages、dealloc_pages 等函数设置物理页面，使用 phys_to_virt、alien_4k 等函数控制虚拟地址和物理地址。

此外，系统需要在内核态和用户态调用提供接口 map_so 来完成各地址空间 vDSO 的初始化。在内核空间编写 vdso_init 函数进行初始化并记录内核 vDSO 区域加载位置，在内核初始化阶段调用此函数完成内核 vDSO 初始化。在用户地址空间初始化时，需要在加载用户程序时构造的地址空间中调用 map_so，同时还需要在用户程序的 auxv 向量中的 SYSINFO_EHDR 位设置为 vDSO 的基址，从而能够让用户程序通过 getauxval 函数拿到 vDSO 基址，从而调用提供接口 init_vdso_vtable 加载 vDSO 中的共享代码。

为了验证 vDSO 移植的正确性，本文还在 Starry-OS 中使用 vipc 库进行验证。由于 vipc 本身依赖 vDSO 提供的共享机制，因此其测试程序能够正确运行，说明新的 vDSO 框架已经能够在 Starry-OS 上正常工作。

4.4.3 vsched2 接口实现

在完成 vDSO 移植后，下一步工作是实现 vsched2 所要求的操作系统接口，并逐步替换 Starry-OS 原有调度机制。vsched2 本身并不直接依赖具体操作系统实现，而是通过 trait 接口抽象底层能力。因此，Starry-OS 需要为其原有任务模型实现对应接口，使调度框架能够正确访问任务状态与系统资源。

在当前设计中，axtask 中的任务对象需要被进一步适配为 vsched2 中的“任务”模型。除了原有线程上下文外，还需要增加协程状态、优先级以及运行模式等信息，从而支持统一调度。

同时，Starry-OS 还需要实现 vsched2 需要的依赖接口，包括：

1. 任务定义；
2. 栈管理；
3. 特权级管理；
4. 地址空间管理；

5. Trap 处理；

6. 多处理器管理。

在这些依赖接口中，任务定义由 `axtask` 结构体实现，通过 `axtask` 本身具有的属性实现；栈管理中由 `alloc::alloc` 和 `alloc::dealloc` 实现栈空间分配和回收；特权级管理参考 `Starry-OS` 的系统调用与返回的实现，通过以特殊的系统调用号发起系统调用和手动设置栈后调用 `ret` 返回用户态实现；地址空间管理通过设置根页表寄存器实现，即 `RISC-V` 架构中的 `satp` 寄存器；Trap 处理中通过将 `Starry-OS` 原有的 Trap 处理逻辑封装成任务实现；多处理器管理在 `Starry-OS` 中有功能相同的内容，可以直接复用其函数。

此外，还需要再 `Starry-OS` 中实现与 `vsched2` 的约定功能。通过修改 Trap 部分的依赖 `axcpu` 中的 Trap 处理向量来实现约定的参数传递；外部中断的处理保留原来 `Starry-OS` 的处理逻辑，但在处理返回时跳转到调度模块 `kschedule` 中；线程上下文保存通过修改用户库实现，在其中增加线程上下文保存的相关逻辑。

目前，共享调度器 `vsched2` 与 `Starry-OS` 的适配工作已经完成实现，测试工作部分完成，缺少一些调试工作。后续将逐步完成测试与调试，并在 `Starry-OS` 上进行进一步性能验证。

第 5 章 实验与分析

5.1 vDSO 测试

为了验证本文移植的 vDSO 机制能够在 Starry-OS 上正常工作，本文选择使用 vipc 库进行功能验证。vipc 是基于 vdso_crate_template 实现的跨地址空间通信库，其运行依赖于 vDSO 提供的共享代码与共享数据机制，因此能够作为 vDSO 是否正确移植的重要验证手段。此外，vsched2 库后续计划加入 vipc 相关内容，因此使用 vipc 库进行验证对后续工作也有促进作用。

vipc 库除 vdso_crate_template 需要的用于内存页面分配与映射相关的依赖接口外，没有其他的依赖接口，因此可以直接调用 vipc 库提供的提供接口。由于 vipc 库提供的测试代码中，依赖 tokio 的用户运行时进行测试，而 Starry-OS 中不应该依赖 tokio 运行时，因此需要参考 vipc 库的测试代码编写使用 Starry-OS 的用户库的测试，同时保证测试逻辑尽量相同，同时能更多的验证 vDSO 功能正确。

编写的测试代码的内容如表 5-1 所示：

表 5-1 基于 Starry-OS 编写的 vipc 库测试内容表

测试	具体内容
test_basic_ipc	同进程内发送单个消息，验证 vDSO 基本功能
test_batch_ipc	同进程内发送 32 个消息，验证顺序正确
test_reply_ipc	同进程内发送并回复，验证双向消息传递
test_map_operations	同进程内对元数据操作
test_fork_ipc	跨进程发送 32 个消息，验证 vVAR 正确
test_fork_mt_ipc	跨进程且跨线程发送 32 个消息，验证 vVAR 数据线程安全
test_signal_ipc	跨进程发送信号，验证使用信号进行 IPC 正确
test_multi_fork_ipc	多个进程间发送消息，验证多 CPU 时 vDSO 映射正确

在 RISC-V 64 的 QEMU 8 模拟器中，以 4 核环境运行。去除编译时和初始化时日志后，运行结果如图 5-1 所示：

```
arch = riscv64
platform = riscv64-qemu-virt
target = riscv64gc-unknown-none-elf
build_mode = release
log_level = warn
backtrace = true
smp = 4

Boot at 2026-05-23 13:33:00.016239500 UTC

[ 0.857065 axnet:131] No vsock device found!
[ 0.859357 axdisplay:26] No display device found!
VDSO and vVAR initialized:
  VVAR at 0x00000000000102000 (size: 0x181a000)
  VDSO at 0x00000000000191c000 (size: 0x9000)
Welcome to Starry OS!
SHLVL=1
HOME=/root
PWD=/

Use apk to install packages.

vipc_test start
=== test_basic_ipc ===
test_basic_ipc PASSED
=== test_batch_ipc ===
test_batch_ipc PASSED
=== test_reply_ipc ===
test_reply_ipc PASSED
=== test_map_operations ===
test_map_operations PASSED
=== test_fork_ipc ===
test_fork_ipc PASSED
=== test_fork_mt_ipc ===
test_fork_mt_ipc PASSED
=== test_signal_ipc ===
test_signal_ipc PASSED
=== test_multi_fork_ipc ===
test_multi_fork_ipc PASSED
vipc_test: ALL TESTS PASSED
```

图 5-1 使用 vipc 库验证 vDSO 运行结果图

可以看到，所有测试都能通过，说明 vipc 在 Starry-OS 中能正常运行，从而说明 vdso_crate_template 库的移植是有效的，vDSO 机制已经能够在 Starry-OS 中正常运行。

5.2 vsched2 测试

目前，Starry-OS 已经实现了共享调度器 vsched2 所要求的 trait 接口，包括上下文切换、地址空间切换、栈分配以及 Trap 处理相关接口。这些接口在 RISC-V 64 环境下，Rust toolchain 版本为 nightly-2025-12-12 的环境下能够通过编译。

由于完整调度流程与用户态协程运行机制仍在进一步完善，目前尚未完成全部功能测试与性能测试。后续工作将重点验证以下内容：

- Trap 处理任务的任务转换

- 特权级切换前后的任务正确性
- 栈共享与栈池机制的稳定性
- 高并发场景下的调度性能

后续还将进一步构建高并发异步场景测试，并与 Starry-OS 原有调度机制进行对比分析，从而验证统一调度框架在系统吞吐量、平均等待时间以及上下文切换开销等方面的性能表现。

结 论

本文围绕用户态调度与内核态调度相互分离产生的调度对象不统一、调度层次分离、任务切换路径割裂三个问题，参考 AsyncOS 的设计，实现了共享调度器 vsched2，并在 Starry-OS 上开展了适配与验证工作。

本文参考 AsyncOS 的统一任务思想，将线程与无栈协程统一抽象为“任务”，并以任务作为调度基本单位，设计并实现了跨特权级与跨地址空间的统一调度模型 vsched2。

在具体实现上，本文设计了基于状态机的调度循环结构，通过外层汇编循环与状态跳转方式，实现了 trap_entry、kschedule、uschedule 与 run_task 等核心状态之间的切换。该设计避免了传统函数调用链对固定栈结构的依赖，使调度器能够在协程共享栈与线程独占栈之间安全切换。

在系统适配方面，本文部分完成了共享调度器 vsched2 与 Starry-OS 的接口适配工作，包括vDSO 机制实现、任务模型适配、上下文切换接口、Trap 处理接口以及栈管理接口等内容。目前，vDSO 机制已经通过 vipc 库完成验证，vsched2 已经能够在 Starry-OS 中完成编译，并初步替换部分原有调度逻辑，但完整统一调度流程与性能测试仍在进一步完善中。

与传统线程调度器相比，vsched2 不再仅以线程作为调度对象，而是支持协程、线程以及 Trap 处理任务共同参与调度，从而能够更好地适应高并发异步场景。此外，vsched2 能够与操作系统相分离，能够实现兼容不同操作系统，实现协程、线程、进程统一调度，为未来在操作系统中进一步研究统一调度、用户态运行时以及跨特权级协同调度机制提供了基础

由于时间限制，本文仍存在一些尚未完成的工作。vsched2 的依赖接口还未进行功能性验证与性能测试。此外，后续还可以进一步研究动态优先级调整与负载均衡策略。

参考文献

- [1] Meng Z, Guo Y, Sun C, et al. Achieving consistent low latency for wireless real-time communications with the shortest control loop[C]//Proceedings of the ACM SIGCOMM 2022 Conference. New York: Association for Computing Machinery, 2022: 193-206.
- [2] 刘健, 黄才胜. 基于协程的高并发架构研究[J]. 数字技术与应用, 2018, 36(04): 85-86. DOI:10.19695/j.cnki.cn12-1369.2018.04.43.
- [3] Oracle. Project Loom: Virtual threads and structured concurrency[EB/OL]. [2026-05-15]. <https://openjdk.org/projects/loom/>
- [4] Node.js Foundation. The Node.js event loop[EB/OL]. [2026-05-15]. <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick>
- [5] AsyncModules. async-os[CP/OL]. [2026-05-15]. <https://github.com/AsyncModules/async-os>.
- [6] 廖东海. ReL4: 高性能异步微内核设计与实现[D]. 北京: 北京理工大学, 2025.
- [7] 赵方亮. 基于软硬协同的任务调度和中断响应研究[D]. 北京: 清华大学, 2025.
- [8] Starry-OS. Starry-OS[CP/OL]. [2026-05-15]. <https://github.com/Starry-OS/Starry-OS>.
- [9] rosy233333. vdso_crate_template[CP/OL]. [2026-05-15]. https://github.com/rosy233333/vdso_crate_template.
- [10] rosy233333. vipc[CP/OL]. [2026-05-15]. <https://github.com/rosy233333/vipc>.
- [11] Lee E A. The problem with threads[R/OL]. EECS Department, University of California, Berkeley, 2006. <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-1.html>.
- [12] Conway M E. Design of a separable transition-diagram compiler[J]. Communications of the ACM, 1963, 6(7): 396-408.
- [13] Moura A L D, Ierusalimsky R. Revisiting coroutines[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 2009, 31(2): 1-31.
- [14] Samson C F. Asynchronous programming in rust: Learn asynchronous programming by building working examples of futures, green threads, and runtimes[M]. [S.l.]: Packt Publishing Ltd, 2024.
- [15] Rust Team. Asynchronous programming in Rust[EB/OL]. [2026-05-15]. <https://rust-lang.github.io/async-book/>.
- [16] Stanza. Async state machines[EB/OL]. [2026-05-15]. <https://www.stanza.dev/courses/rust-async/future-trait/rust-async-state-machines>.
- [17] Jia Y, Tian K, You Y, et al. Skyloft: A general high-efficient scheduling framework in user space[C]//Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles. New York: Association for Computing Machinery, 2024: 265-279.
- [18] The Go Authors. Effective Go: concurrency and goroutines[EB/OL]. [2026-05-15]. https://go.dev/doc/effective_go.
- [19] Calmops. High-performance networking: Tokio runtime explained[EB/OL]. [2026-05-15]. <https://calmops.com/programming/rust/high-performance-networking-tokio-runtime-explained/>.
- [20] Linux Kernel Developers. vDSO[EB/OL]. [2026-05-15]. <https://www.kernel.org/doc/html/latest/admin-guide/abi-stable.html#vdso>.
- [21] Kerrisk M. vdso(7) — overview of the virtual ELF dynamic shared object[EB/OL]. [2026-05-15]. <https://man7.org/linux/man-pages/man7/vdso.7.html>.

- [22] LWN.net. KAISER: hiding the kernel from user space[EB/OL]. [2026-05-15]. <https://lwn.net/Articles/738975/>.
- [23] The Rust Project Developers. Inline assembly[EB/OL]. [2026-05-15]. <https://doc.rust-lang.org/stable/reference/inline-assembly.html>.

致 谢

时光荏苒，大学阶段的学习即将结束。在本次毕业设计与论文写作过程中，我得到了许多老师、同学以及朋友的帮助与支持，在此谨致以诚挚的感谢。

首先，感谢我的指导老师陆慧梅老师、清华大学向勇老师和北京工商大学吴竞邦老师在选题、方案设计以及论文撰写过程中给予的悉心指导与耐心建议。从课题方向的确定到系统实现细节的讨论，各位老师提供了具有启发性的意见，使我能够不断完善设计思路并顺利完成相关工作。

其次，感谢实验室罗熙学长和陈林峰学长在开发与调试过程中给予的帮助。在系统实现过程中，与学长合作的过程让我对操作系统设计有了更深的理解，与大家的交流和讨论对我理解复杂系统设计起到了重要作用。

同时，也感谢在学习与研究过程中接触到的开源社区与相关项目，包括 Rust 生态、Linux 内核文档以及 Starry-OS 文档等资料，它们为本课题的实现提供了重要的理论基础与实践参考。

最后，感谢学校为我提供良好的学习环境和实践平台，使我能够顺利完成本次毕业设计与论文撰写。